ARI5118 - AI Usage Journal

Michael Vella (michael.vella.20@um.edu.mt)

ARI5118 - May 2026

1 Introduction

In this project, the focus was not whether to use AI tools but how to use them deliberately, critically, and transparently. The assignment brief itself frames AI assistance as an expected part of the workflow, shifting the question from permission to accountability. With that in mind, I approached AI tools as a collaborator to think with rather than a system to delegate to. A tool, useful for accelerating drafts, writing code, and surfacing edge cases, but not a substitute for the understanding required to verify, correct, and own every deliverable.

The AI tools used throughout this project were three:

1. **Anthropic's Claude (Sonnet 4.6, Extended Thinking)**: Claude served as the primary AI assistant throughout this project. It was used across most deliverables, from refining and summarising explanations to generating and debugging code. A particularly useful feature was Claude's Projects functionality, which allows related conversations to be grouped together with shared attachments and context that persist across all prompts within the project. This made it straightforward to keep the assignment brief, study notes, and other reference materials consistently available without re-uploading them each session. Ultimately, however, the main reason Claude was selected was due its strong performance on code generation tasks, which was central to producing the simulator and annotated walkthrough deliverables.

2. **NotebookLM**: Used to summarise and cross-reference the key papers collected during the research phase. It was also used to generate an initial draft of the presentation slide deck from the study notes. While that draft was not used directly in the final submission, it served as a useful structural reference from which several ideas and choices were borrowed.

3. **Google Search (AI-assisted)**: Google Search was used for general research and paper discovery. As Google now integrates AI-generated summaries directly into search results, some degree of AI assistance was present in this process, though it was treated as a starting point for locating sources rather than a source of factual claims in its own right.

2 Ethical Considerations

The use of AI tools in an academic context carries responsibilities that extend beyond simply producing correct outputs. Throughout this project, several ethical dimensions were considered.

Data Bias and Accuracy. Large language models such as Claude are trained on vast corpora of text that inevitably reflect the biases present in their sources. In the context of this project, this manifests most concretely in how technical concepts are explained and emphasised. Therefore, every AI-generated explanation was reviewed against primary sources to ensure accuracy and balance. At times, the model produced confident but slightly incorrect statements and these were identified and corrected before inclusion in any deliverable. AI output was treated as a first draft, not a final authority.

Privacy. No personal data, proprietary datasets, or confidential information were submitted to any AI tool during this project. All inputs consisted of academic content, code, and publicly available material. Nevertheless, it is worth acknowledging that prompts submitted to cloud-based models may be retained and used for model improvement depending on the provider's data policy. This was considered when deciding what to include in prompts but given that no sensitive information used during this project, there were minimal risks in this regard.

Academic Conduct. The use of AI tools in this project was approached in accordance with the assignment brief, which explicitly permits AI assistance provided that intellectual ownership remains with the student. All AI-generated content was reviewed, revised, and verified before inclusion. Submitting AI output uncritically, without verification or understanding, would undermine the learning objectives of the assignment.

Hallucinated References and Paper Verification. One of the most well-documented failure modes of large language models is the generation of plausible-sounding but entirely fictitious citations. A model may produce a correctly formatted reference (with a real author's name, a plausible journal, and a believable title) that does not exist. To mitigate this risk, no AI-generated reference was accepted without independent verification. Each key paper was located manually and confirmed to exist before being cited. The PDF of each verified paper was downloaded and included in the `further_reading/` folder as required, serving as evidence that every cited work is real and accessible.

3 Methodology

An attempt was made to integrate AI assistance into each of the deliverables, though the degree and nature of that integration varied deliberately depending on the task.

Study Notes. The initial study notes were written manually, drawing from multiple academic papers and online sources, without AI involvement. This was a deliberate choice as I wanted to engage directly with source material as I feel that this is the most reliable way to build genuine understanding of technically dense content, and outsourcing that step to an AI would have undermined the learning the assignment is designed to produce. Once the notes were complete, Claude was used as an enhanced summarisation tool to condense the material into the Quick Reference section. Because the underlying writing was entirely my own, the model had accurate, well-structured content to work from, and its output was subsequently reviewed, revised, and refined before inclusion in the final deliverable.

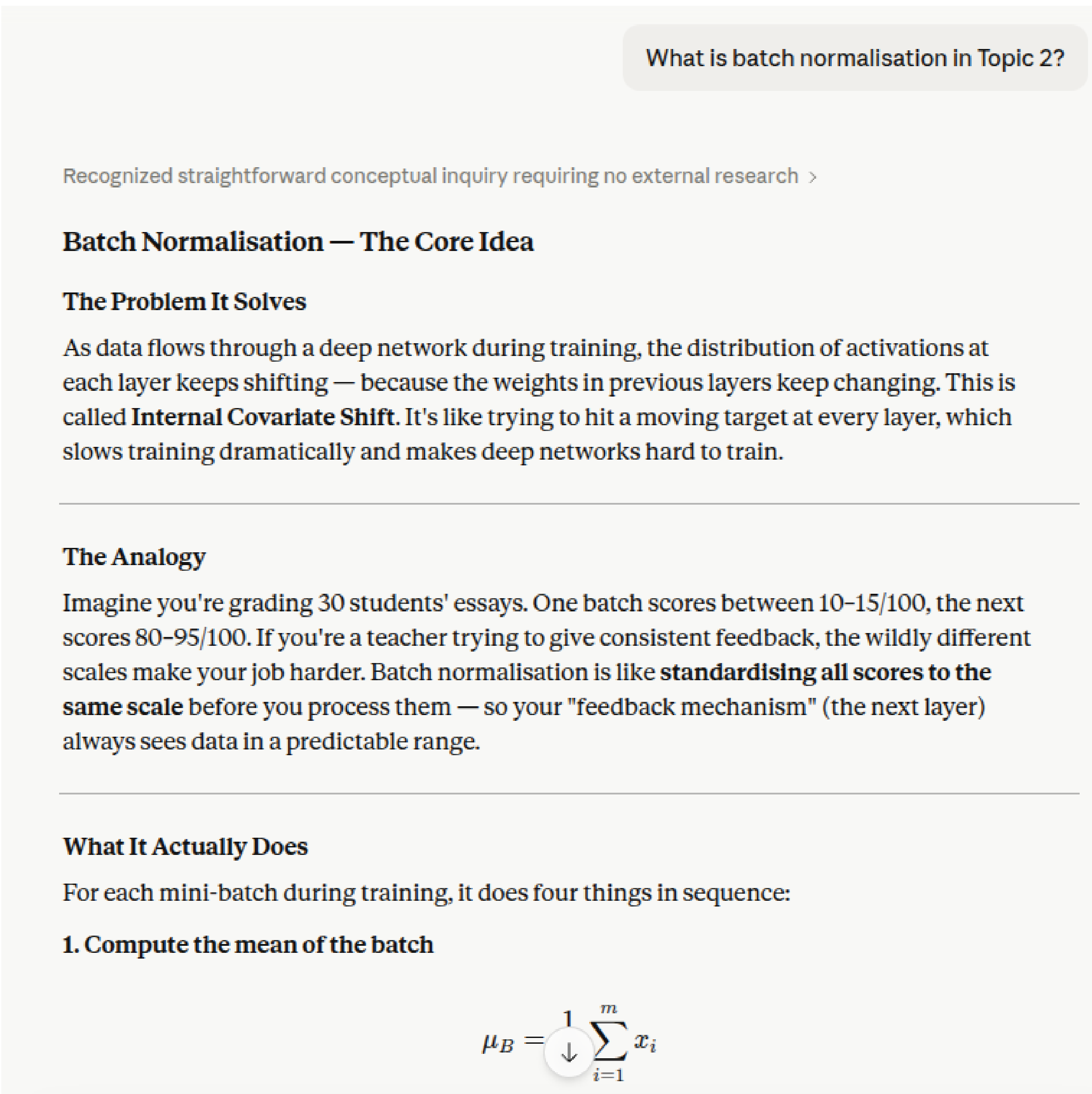
Adversarial Quiz. Claude was used to generate an initial draft of the quiz using the study notes as context. The output was not satisfactory. Several questions were too straightforward to qualify as adversarial, and others addressed aspects of the topic that fell outside the scope of the study notes. The draft was therefore used only as a rough structural reference, with all questions rewritten or replaced manually to meet the depth and specificity the deliverable required.

Presentation Slides. NotebookLM was used to generate an initial slide deck from the study notes. While the output was broadly reasonable in structure, a number of slides required significant revision. A critical limitation encountered was that NotebookLM does not permit direct editing of the generated slide content, making it impossible to apply the necessary corrections within the tool. Considerable time was spent attempting to work around this constraint before the decision was made to redesign the slides from scratch. The final slides were produced independently, borrowing only selected structural ideas from the original generated deck.

Simulator and Notebook Walkthrough. For both the interactive simulator and the annotated code walkthrough, Claude was used to generate an initial baseline implementation. In both cases, the generated code served as a functional starting point rather than a finished product. Each was subsequently reviewed, restructured, and extended to meet the standards required and to ensure the code was fully understood before submission.

4 Prompts and Responses

Upon choosing a topic, I was drawn to Topic 2 but was unfamiliar with batch normalisation at the time. Before committing, I used Claude to obtain a high-level summary of the topic to assess whether it was something I felt confident pursuing. As shown in Figure 1, the summary Claude provided was clear and accessible, giving me enough of an initial understanding to make an informed decision. This interaction directly influenced my choice to register for Topic 2.



What is batch normalisation in Topic 2?

Recognized straightforward conceptual inquiry requiring no external research >

Batch Normalisation — The Core Idea

The Problem It Solves

As data flows through a deep network during training, the distribution of activations at each layer keeps shifting — because the weights in previous layers keep changing. This is called **Internal Covariate Shift.** It's like trying to hit a moving target at every layer, which slows training dramatically and makes deep networks hard to train.

The Analogy

Imagine you're grading 30 students' essays. One batch scores between 10–15/100, the next scores 80–95/100. If you're a teacher trying to give consistent feedback, the wildly different scales make your job harder. Batch normalisation is like **standardising all scores to the same scale** before you process them — so your "feedback mechanism" (the next layer) always sees data in a predictable range.

What It Actually Does

For each mini-batch during training, it does four things in sequence:

1. Compute the mean of the batch

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i$$

Figure 1: Asking Claude what batch normalisation is.

At the initial stage of the project, after registering for Topic 2, Claude was prompted to suggest key papers that contributed to the development and advancement of the topic. As shown in Figure 2, a list of relevant papers was returned, which provided a useful starting point and reduced the time that would otherwise have been spent searching manually. Not all of the suggested papers were selected for inclusion. Moreover, each candidate was independently verified by locating the original source, confirming the citation details, and reading the abstract to ensure the paper matched the description provided.

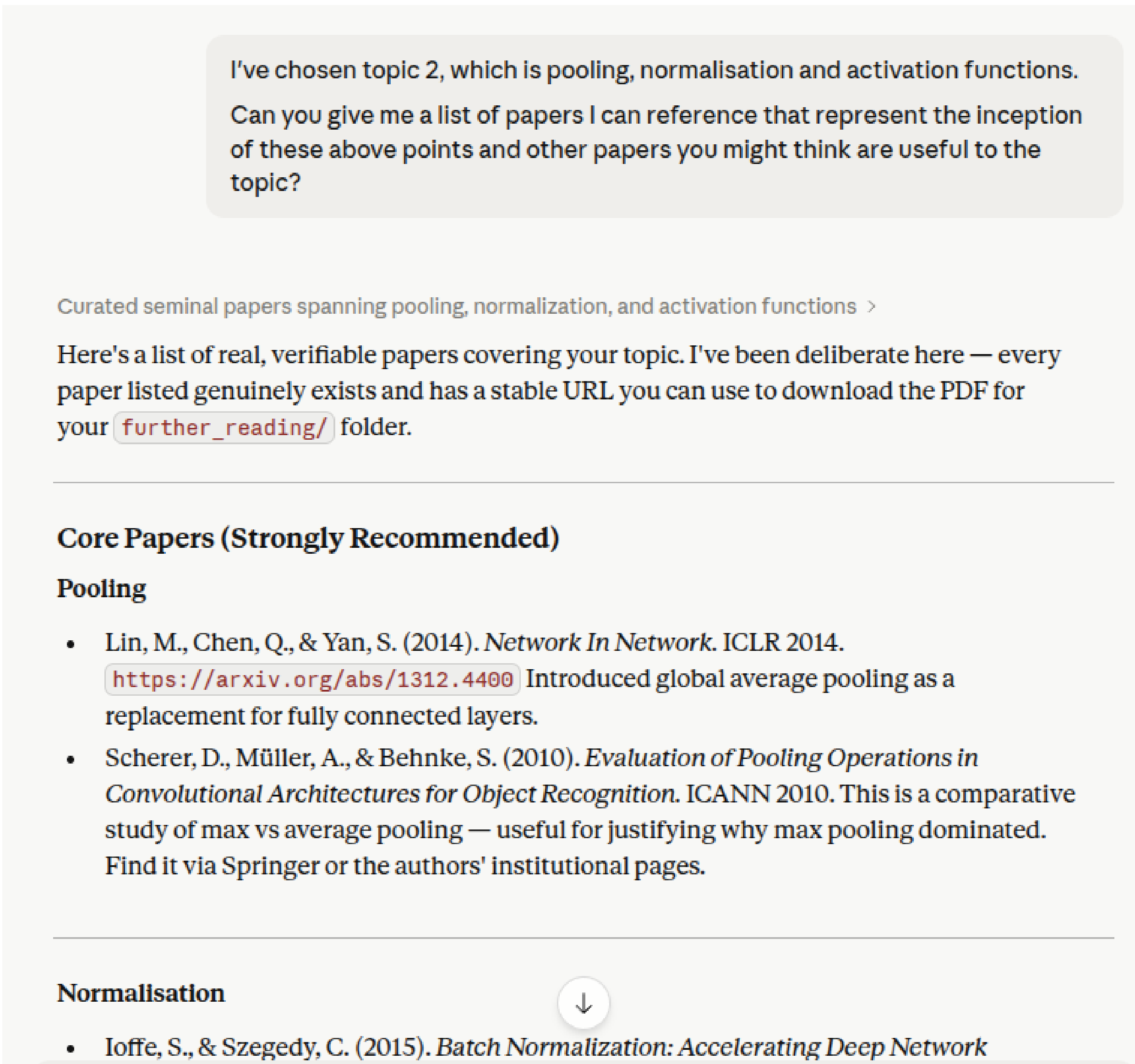


Figure 2: Requesting a list of key papers focused around topic 2 (pooling, normalisation and activation functions).

As outlined in the Methodology section, the study notes were written manually from primary sources. Once a draft was complete, the notes were submitted to Claude with a prompt asking it to identify any concerns, inconsistencies, or areas that were unclear or potentially incorrect. This proved to be a consistently valuable step as Claude raised legitimate points that required revision (whether gaps in explanation, ambiguous phrasing, or subtle technical inaccuracies - see Figure 3). The necessary amendments were made independently before the notes were considered final. The revised notes were then submitted to Claude a second time with a request to produce a summarised version, which formed the basis of the Quick Reference section (see Figure 4). Using the model as a critical reviewer rather than a generator meant that the underlying content remained entirely my own, while the AI contributed meaningfully to improving its accuracy and clarity.

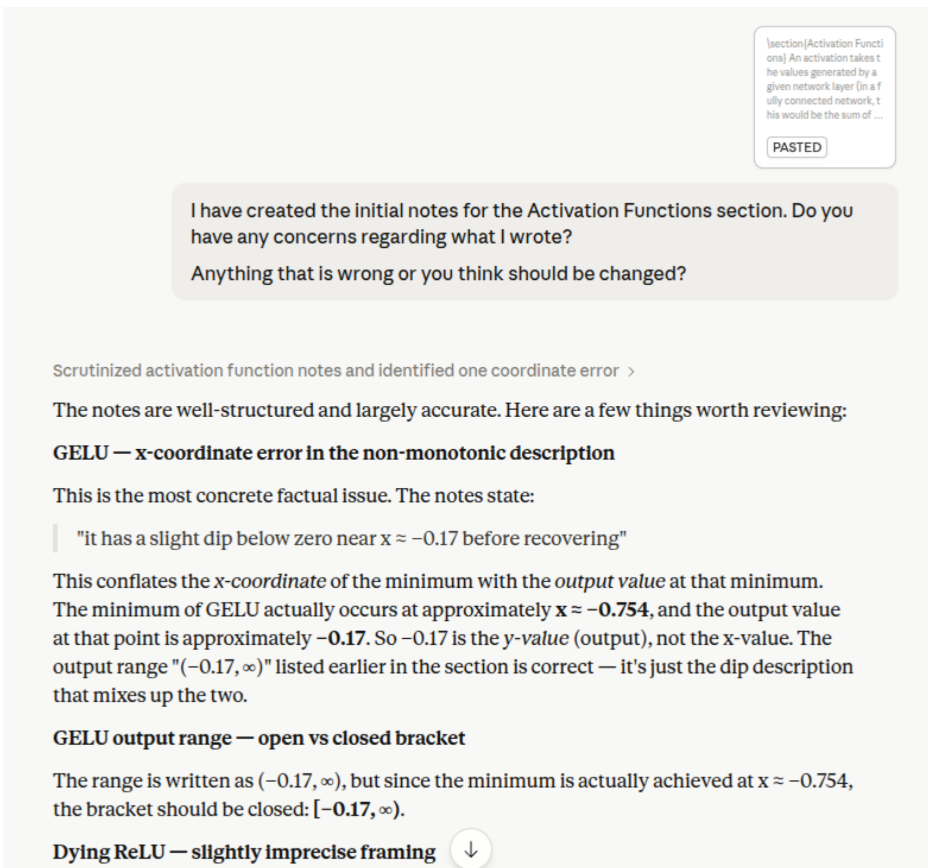


Figure 3: Asking Claude to review the notes I had written for the Activation Functions section.

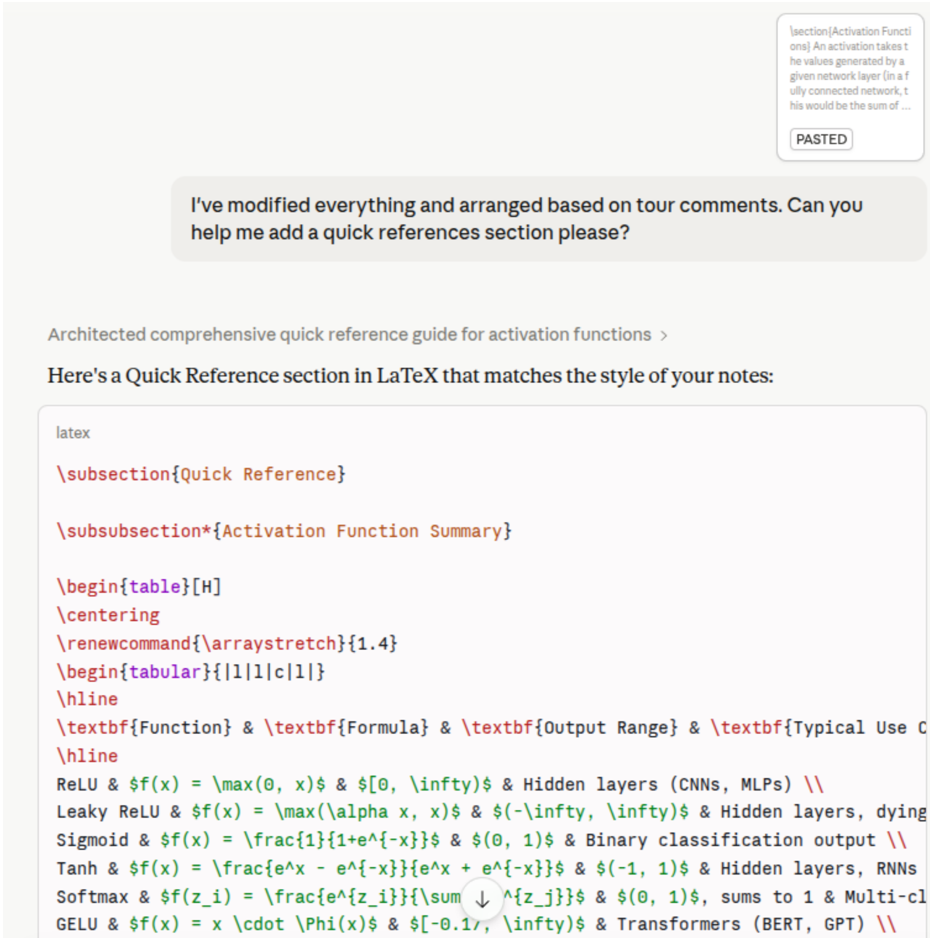


Figure 4: Asking Claude to summarise the main activation function notes into a Quick Reference section.

AI assistance contributed significantly to the development of the annotated code walkthrough, saving considerable time that would otherwise have been spent on initial implementation. As shown in Figure 5, Claude was prompted to generate an initial design for the notebook pipeline. Rather than producing only a structural outline, Claude returned a fully implemented walkthrough, complete with code and accompanying narrative. While this provided a strong starting point, not all sections were accurate or very elaborative upon review. These sections were identified through careful reading and revised using my own judgement and understanding of the topic, ensuring that the final notebook reflected a genuine comprehension of the material rather than unverified AI output.

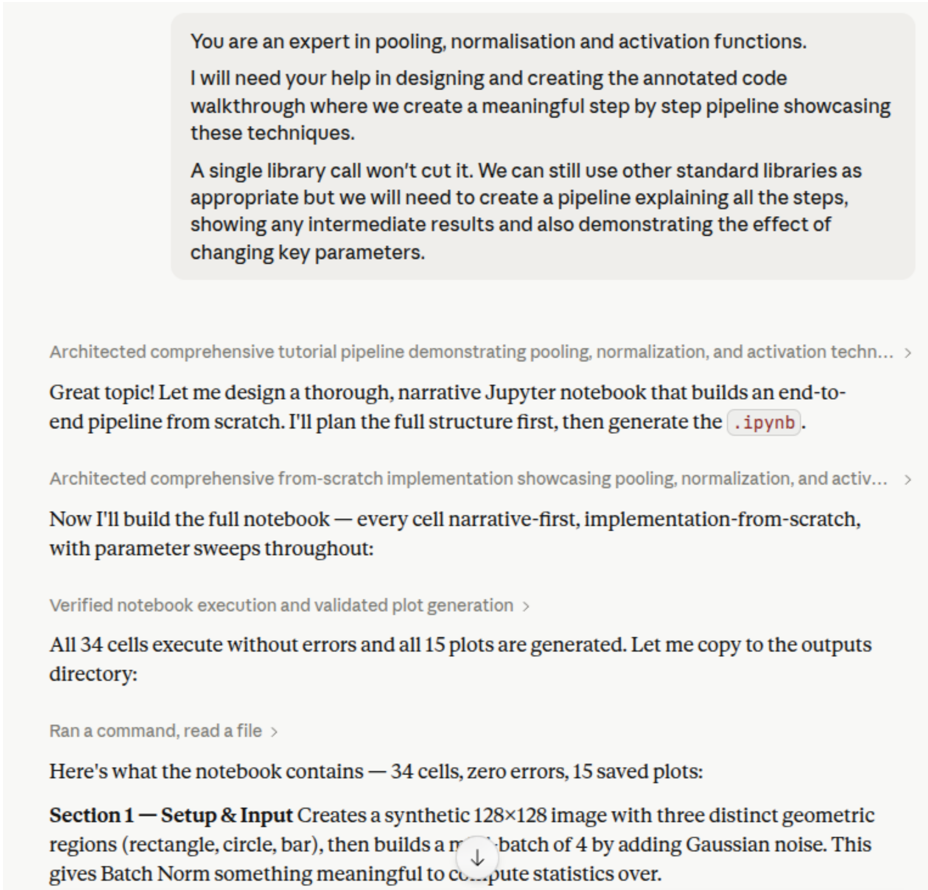


Figure 5: Asking Claude to design an annotated code walkthrough to showcase pooling, batch normalisation and activation functions.

A similar approach was taken for the interactive simulator. As shown in Figure 6, Claude was prompted to generate an initial design for the application. A Streamlit app was produced from the outset, though it did not run correctly out of the box and required a round of debugging before it was functional. Once working, it served as a solid starting point from which the application could be extended. Subsequent attempts to refine and amend the simulator using Claude proved less successful. Without access to the full codebase as context, the model's suggestions were often misaligned with the existing implementation and introduced further issues rather than resolving them. This highlighted an important limitation of using AI for iterative development on larger codebases as it is most effective when given complete context, and progressively less reliable as that context becomes harder to supply. As a result, the later refinements were made manually, which reinforced the importance of having a genuine understanding of the code rather than relying on the model to maintain it.

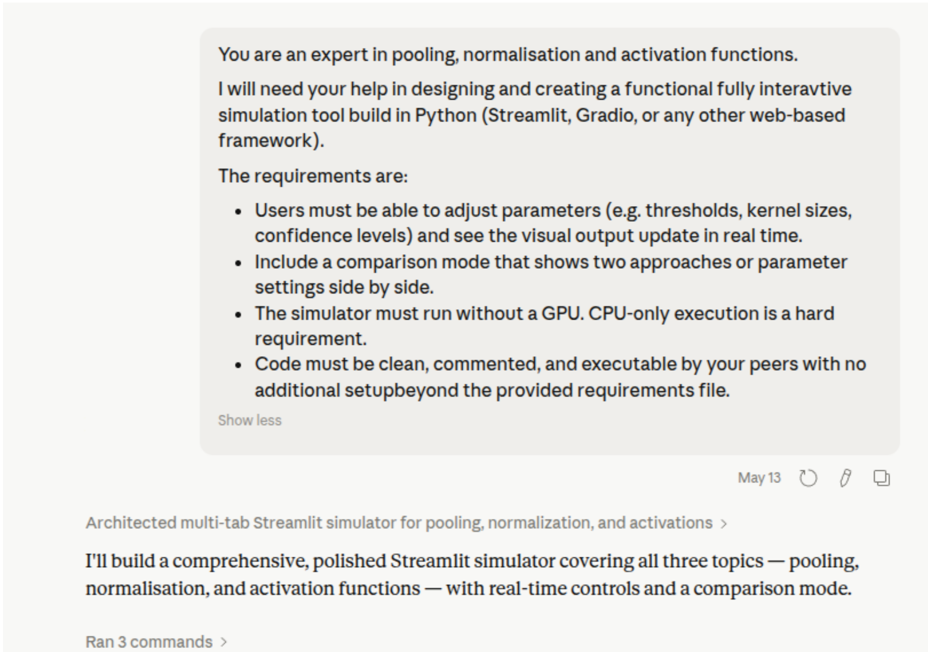


Figure 6: Asking Claude to design the initial pooling, batch normalisation and activation function simulator.

5 Improvements and Contributions

While AI assistance was attempted across most deliverables, there were two areas in particular where it made a tangible and positive contribution to the final output.

Summarisation of Study Notes. The most immediately useful contribution AI made was in condensing the study notes into the Quick Reference section. Writing comprehensive notes from primary sources is a time-intensive process, and distilling that material into a concise, well-organised cheat sheet is an equally demanding task. By providing Claude with the completed notes as context, the model was able to identify the most important concepts, equations, and parameter ranges and present them in a structured format. The output required revision, but the underlying structure it produced was sound and saved considerable time compared to building the cheat sheet from scratch. The result was a more complete and consistently formatted Quick Reference section.

Code Generation for the Simulator and Walkthrough. The most substantive contribution AI made was in generating the initial implementations for both the interactive simulator and the annotated notebook walkthrough. Writing functional, well-structured code from a blank file is often the most time-consuming part of any technical deliverable, and having a working baseline to build from significantly accelerated the development process. Claude was able to scaffold the core logic and produce readable code that could be understood, tested, and extended. This freed up time to focus on the higher-level decisions (what parameters to expose, how to structure the comparison mode, how to narrate the walkthrough) rather than on boilerplate implementation. In both cases, the generated code was not used verbatim and it was reviewed line by line, corrected where necessary, and meaningfully extended before submission. The AI's contribution here is best described as accelerating the groundwork so that more effort could be directed toward refinement and understanding.

6 Individual Reflection

Reflecting on my experience using AI throughout this project, the clearest conclusion is that AI is a genuinely useful tool but only when used with discipline. Knowing when to reach for it and when to work independently is a skill in itself and equally important is the willingness to discard AI output without hesitation when it falls short, rather than investing time trying to salvage something that was never going to meet the required standard. Above all, the habit of independently verifying everything the model produces is non-negotiable. Without it, errors go undetected, hallucinations go uncorrected, and (in my opinion) nothing is genuinely learnt.

7 References and Resources Used

AI tools used:

1. **Anthropic's Claude (Sonnet 4.6, Extended Thinking)**.
2. **NotebookLM**.
3. **Google Search (AI-assisted)**.

For any other (non-AI) resources referenced, refer to the additional reading and key papers sections found inside the study notes.